Which Wrong Pages Should an Event Linker Train On?

Negative sampling for the retrieval stage of Event Linking

Rodrigo Quezada
COMPSCI 2242, Fall 2026

Draft, November 9, 2026

Abstract

A retriever learns from contrast: during training it sees the correct item next to a few wrong ones, the negatives, and the choice of negatives decides what it learns. We study that choice for event linking, where a mention in text must be matched to one of 5.87 million Wikipedia event pages and the hardest wrong pages describe the same event in another year. At a fixed budget we compare four samplers for the EVELINK retriever: in-batch negatives, uniform draws from the top 1,000 candidates, the highest-scoring candidates, and ReSa2, a two-stage sampler that prefers candidates close to the correct page. ReSa2 reaches 93.3% recall at ten against 92.2% for uniform draws and 69.6% for in-batch negatives, and puts the correct page first 69.5% of the time against 64.3%. We measure what each sampler selects, prove the loss bound and three counting facts in Lean 4, and show that the hardest negatives can make training collapse.

1 Introduction

EVELINK is a system that reads a mention of an event and tries to find the Wikipedia page for that event. Yu and colleagues introduced it in 2023 [Yu et al.(2023)]. You give it a phrase like "won the Rising Star award," along with the sentence around it. EVELINK uses that context to work out which award the writer is talking about.

Take this sentence. "He won the Rising Star award in 1996, in his first season with Adelaide."

The page we want is "1996 AFL Rising Star." A page about the 2004 Indian Ocean earthquake is easy to rule out. The page for the 1998 AFL Rising Star is harder. It is the same award. The year is what tells us which page to link to. The Event Linking paper calls this a time error; its other common error, a hierarchy error, confuses an event with a parent event or a sub-event. In both cases the wrong page shares much of its wording with the correct page.

EVELINK makes that decision in two stages (section 2). First, a retriever searches Wikipedia and pulls out 100 candidate pages. Then a reranker reads the mention alongside each candidate and puts the pages in a new order. If the retriever leaves out the 1996 page, the reranker cannot bring it back.

That is where this project starts. During training, the retriever sees the correct page alongside a few wrong pages. I wanted to know which wrong pages would help it learn to put the right one higher. Two questions guide the paper:

1. How does ReSa2 [Li et al.(2025)], a two-stage sampler of wrong pages, change the inclusion probability of each candidate, the coverage of the candidate pool, and the variation between runs?
2. With a fixed compute budget, does ReSa2 put the correct page higher (recall at ten) than the original sampling?

The contributions are: (1) a fixed-budget comparison of four negative samplers for the EVELINK retriever, with three seeds each; (2) a measurement of what each sampler selects: inclusion probability, coverage, and variation between runs; (3) the loss bound and three counting facts about possible false negatives, proved in Lean 4; and (4) a description of a training collapse that appears with hard negatives at a high learning rate, with a gradient argument for its cause and a test that shows it does not recover. The code, the pools, and the logs are in the project repository.

2 Event linking: task, system, and evaluation

Before the samplers, the task itself: what event linking is, how EVELINK does it, and how both are scored.

Task. Let $\mathcal{E}$ be the set of Wikipedia pages that describe events; in the data of Yu et al., $|\mathcal{E}| =$ 5,870,755. A *mention* $m = (d, [a, b])$ is a document d and a span $[a, b]$ of its tokens that refers to an event. Event linking is the function

$$\text{link} : m \longmapsto e_m^* \in \mathcal{E} \cup \{\text{NIL}\}, \tag{1}$$

where e_m^* is the page about the event the span refers to, and NIL means that no page describes it. The Wikipedia data has no NIL mentions; the New York Times data has them. The goal is to recover e_m^* for every mention, using the span and the words around it.

Data. Training and evaluation pairs (m, e_m^*) come from links that Wikipedia editors made: when an article's text links a span to an event page, that span is a mention and the target page is its answer. The released split has 66,426 training, 16,692 validation, and 19,266 test mentions, each labeled as a verb or a nominal and, for verbs, as seen, unseen form, or unseen (whether the verb and its target appear in training). The New York Times set (1,762 mentions) tests transfer to another domain.

System. Exact search over 5.87 million pages with an expensive model is too slow, so EVELINK, like BLINK before it, works in two stages:

$$\text{retrieve:} \quad R_K(m) = \text{top-}K_{e \in \mathcal{E}}\ s(m, e), \tag{2}$$

$$\text{rerank:} \quad \hat{e}_m = \underset{e \in R_K(m)}{\arg\max}\ r(m, e). \tag{3}$$

The retriever scores every page with a cheap function s, the dot product of two vectors computed separately (section 4), and keeps the $K = 100$ best. The reranker reads the mention and each of those K pages together, with an expensive function r, and returns the best one. Equation 3 is also the reason this paper is about the retriever. If $e_m^* \notin R_K(m)$, the reranker never sees the right page.

Evaluation. The full system is judged by accuracy,

$$\mathrm{Acc} = \frac{1}{|V|} \sum_{m \in V} \mathbf{1}[\hat{e}_m = e^*_m] \,, \tag{4}$$

over an evaluation set V, reported overall and per class (verb, nominal, and their subclasses). The retriever alone is judged by whether the correct page survives retrieval,

$$\mathrm{Recall@}K = \frac{1}{|V|} \sum_{m \in V} \mathbf{1}[e^*_m \in R_K(m)] \,, \tag{5}$$

which bounds the accuracy of any reranker placed after it. This paper changes only how the retriever is trained, so its main measure is Recall@10 on the validation set, with Recall@1 (the correct page first) and Recall@100 next to it. The released EVELINK system reaches 82.87% accuracy on the Wikipedia test set and 55.33% on the New York Times set; the released retriever alone has Recall@10 of 77.7% on the validation set.

Where the errors are. Yu et al. report two error types that account for most mistakes. A *time error* links to the page of the same recurring event in another year (the 1998 Rising Star for a 1996 mention). A *hierarchy error* links to a parent or a sub-event (the whole war for one battle). Both wrong pages share most of their words with the correct page, so the retriever scores them high. These are the pages it has to learn to rank below the correct one, and the samplers of section 5 differ in how often they show them to it.

3 Related work

Event linking. Yu et al. [Yu et al.(2023)] define the task, release the Wikipedia and New York Times data, and build EVELINK on the BLINK bi-encoder and cross-encoder of Wu et al. [Wu et al.(2020)]. They train the bi-encoder with in-batch negatives and retrieve 100 candidates for the cross-encoder. This paper keeps their model and data and changes only how the bi-encoder's negatives are chosen.

Negatives for dense retrieval. In-batch negatives [Karpukhin et al.(2020)] cost nothing extra, but most of them are easy. Hard-negative mining takes negatives from the retriever's own top results instead. ANCE [Xiong et al.(2021)] refreshes them during training; RocketQA [Qu et al.(2021)] filters them with a cross-encoder, because many top results are unlabeled positives (false negatives). The top-score sampler in this paper is that idea without the filter. SimANS [Zhou et al.(2022)] samples negatives with a Gaussian weight centered on the positive's score, the same weight ReSa2 uses in its first stage.

ReSa2. Li et al. [Li et al.(2025)] sample in two stages: a weighted draw that prefers pages whose score is close to the positive's, then a similarity ranking among the survivors. The sizes used here (pool 1,000, 500 survivors, top 75, 7 negatives) are the ones in their released code. This paper measures what those two stages do to the inclusion probability of each page and compares the result with uniform and top-score sampling at a fixed budget.

4 The retriever and its training loss

The retriever of equation 2 uses two BERT encoders. One reads the mention and its context. The other reads the Wikipedia page. Each encoder turns its input into a vector with 1,024 numbers. I call the mention vector u_m and the page vector v_e.

To score a page, I take the dot product. I multiply the first number in one vector by the first number in the other, do the same for the rest, and add up the products:

$$s(m, e) = u_m \cdot v_e = \sum_{j=1}^{1024} u_{m,j}\, v_{e,j}. \tag{6}$$

The retriever puts pages with higher scores first. The two encoders work separately, so I can compute the page vectors ahead of time. When a new mention comes in, I encode it once and compare that vector with the stored page vectors.

During training, I give the retriever the correct page and seven wrong pages. I call the wrong pages *negatives*. If the correct page scores s^+ and the negatives score s_1 through s_7, the loss is

$$L = -\log \frac{e^{s^+}}{e^{s^+} + \sum_{i=1}^{7} e^{s_i}}. \tag{7}$$

The fraction is the correct page's share of the total after I exponentiate the eight scores. When the correct page gets almost all of that share, the loss is close to zero. When a wrong page scores about as high as the correct one, it takes a bigger share and the loss goes up.

To see the difference, suppose the correct page scores 10 and there is just one negative. If that negative scores -10, the loss is about two billionths. If it scores 9.5, the loss is about 0.47. The correct page has the same score in both cases. All I changed was the wrong page.

The math puts a bound on this. Write $g_i = s^+ - s_i$ for the gap by which negative i scores below the correct page.

Lemma 1 (Easy negatives add little)**.** *For any $n \geq 1$ and real scores $s^+, s_1, \ldots, s_n$, the loss of equation 7 with n negatives satisfies*

$$L = \log\Bigl(1 + \sum_{i=1}^{n} e^{-g_i}\Bigr) \leq \sum_{i=1}^{n} e^{-g_i}, \qquad \textit{and} \qquad L \leq n\, e^{-g} \quad \textit{if every } g_i \geq g. \tag{8}$$

Proof. Divide the numerator and the denominator of the fraction in equation 7 by e^{s^+}: $L = -\log\bigl(1/(1+\sum_i e^{s_i - s^+})\bigr) = \log(1+\sum_i e^{-g_i})$. For $x \geq 0$, $\log(1+x) \leq x$, so the first inequality holds. If every $g_i \geq g$ then each $e^{-g_i} \leq e^{-g}$, and the second follows. □

A negative g points below the correct page adds at most e^{-g} to the loss. At $g = 20$ that is about 2×10^{-9}; at $g = 0.5$ it is about 0.61. Lemma 1 is proved in Lean 4 (section 5.1).

I can also ask how much the loss changes if I raise one negative's score.

Lemma 2 (The push on a negative)**.** *The derivative of L with respect to the score of negative i is that negative's share of the softmax,*

$$p_i = \frac{\partial L}{\partial s_i} = \frac{e^{s_i}}{e^{s^+} + \sum_{j=1}^{n} e^{s_j}} \in (0, 1), \tag{9}$$

and $\partial L/\partial s^+ = -\sum_i p_i$.

Proof. Write $Z = e^{s^+} + \sum_j e^{s_j}$, so $L = \log Z - s^+$. Then $\partial L/\partial s_i = e^{s_i}/Z$ and $\partial L/\partial s^+ = e^{s^+}/Z - 1 = -\sum_i e^{s_i}/Z$. □

So if the retriever already puts the earthquake page far below the 1996 award, that page contributes almost nothing to the update. If the 1998 award scores close to the 1996 award, the loss puts more pressure on the model to separate them.

The update itself moves the mention vector.

Corollary 1 (The update on the mention vector)**.** *With the page vectors fixed, the gradient of L with respect to the mention vector is*

$$\nabla_{u_m} L = \sum_{i=1}^{n} p_i \, (v_{e_i} - v_{e^+}), \tag{10}$$

Proof. By the chain rule through equation 6, $\nabla_{u_m} L = \sum_i (\partial L/\partial s_i)\, v_{e_i} + (\partial L/\partial s^+)\, v_{e^+} = \sum_i p_i v_{e_i} - \big(\sum_i p_i\big) v_{e^+}$ by Lemma 2. □

Equation 10 is a weighted sum of the differences between each negative's page vector and the correct page vector. A negative that scores far below the correct page has $p_i \approx 0$ and moves nothing. A negative that scores close to it has a large p_i, and the step is along $v_{e_i} - v_{e^+}$. If the two pages are near-duplicates, that difference is small. Section 6.4 returns to this.

In the experiment, I keep the loss and the number of negatives fixed. I change which seven pages the model sees.

5 How I chose the seven pages

I compared four ways of picking negatives. The first uses the correct pages for the other seven mentions in the same small batch. That is the in-batch baseline. For the other three, I started with the pages that the released retriever ranked highest for that mention. I built a pool of up to 1,000 candidates and excluded the correct page from sampling. Every sampler sees the same input: the pool scores s_i, the score s^+ of the correct page, and the similarity $\sigma_i = v_{e_i} \cdot v_{e^+}$ of each pool page to the correct page.

Uniform sampling gives every page in the pool the same chance, $k/N = 7/1{,}000$. Top-score sampling always takes the seven highest-scoring pages, so its inclusion probability is 1 for those seven pages and 0 for the rest. ReSa2 makes the choice in two stages.

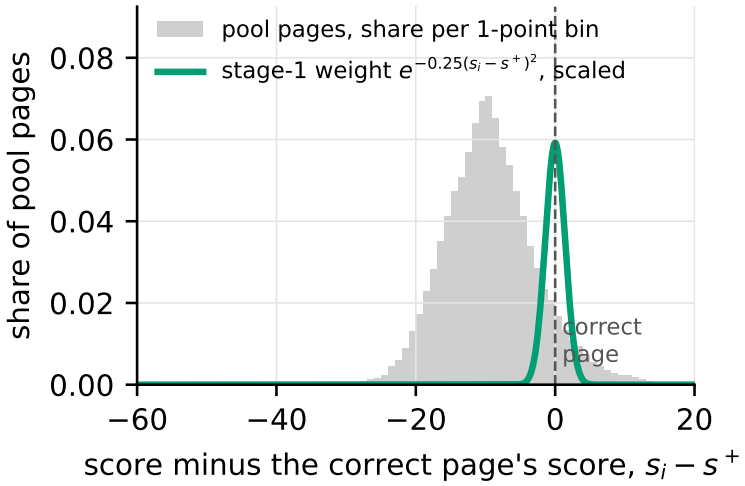


Figure 1: The stage-1 weight of equation 11 (green, scaled) over the distribution of score gaps $s_i - s^+$ in the validation pools (gray; 2,000 mentions × 1,000 pages). Most pool pages score 5 to 20 points below the correct page; the weight is near zero outside ±4 points, so stage 1 is a narrow window around the correct page's score.

First, ReSa2 draws 500 pages without replacement. It gives the most weight to pages that score close to the correct page:

$$w_i = \exp\!\left(-\tau\,(s_i - s^+)^2\right), \qquad \tau = 0.25. \tag{11}$$

It draws one page with probability $w_i / \sum_j w_j$, removes it, and draws again from the rest (successive sampling). The weight drops as the score moves away in either direction. Figure 1 shows the weight over the pool's actual score gaps: with raw scores of about 550 and gaps of tens of points, the weight is near zero outside $|s_i - s^+| < 4$, so in practice stage 1 is a window around s^+, filled up to 500 with almost uniform draws from the rest. Then ReSa2 compares the 500 survivors to the correct page itself. It keeps the 75 pages whose vectors are most similar to the correct page vector and draws seven of those uniformly.

That last draw gives a simple bound. Once a page makes the final 75, it has a 7-in-75 chance of being used. So even if a page labeled as wrong is actually a valid link, I can bound how often ReSa2 trains against it and, with equation 9, bound its expected effect on the loss gradient:

$$\begin{aligned} \mathbb{E}[p_i] &\le P(\text{ReSa2 picks } i) \cdot \frac{e^{s_i}}{e^{s^+} + e^{s_i}} \\ &\le \frac{7}{75} \cdot \frac{e^{s_i}}{e^{s^+} + e^{s_i}}. \end{aligned} \tag{12}$$

5.1 What is proved in Lean

Four claims are formalized in Lean 4 (the files are in Appendix B). The first is the loss bound of equation 8, proved over the real numbers with Mathlib. For the other three, a page is modeled by its score, an integer, and a *possible false negative* is a page that scores above the correct page, $s_i > s^+$.

1. Lemma 1, for any number of negatives with real scores (`loss_eq_log_gaps`, `loss_le_sum_exp_neg_gap`, `loss_le_of_gap_ge`; the Lean proof uses Mathlib's $\log y \le y - 1$).
2. Proposition 1 (`any_sampler_le`) and Proposition 2 (`topscore_is_max`).
3. Proposition 3 (`stage2_share`, checked by computation).

Proposition 1 (A bound for every sampler)**.** *Let the pool contain A possible false negatives. Any selection of k distinct pages from the pool contains at most* $\min(k, A)$ *of them.*

Proof. The selection has k pages, and it is a sub-list of the pool, so it cannot contain more pages above s^+ than the pool does. □

Proposition 2 (Top-score reaches the bound)**.** *The top-score rule selects exactly* $\min(k, A)$ *possible false negatives.*

Proof. Sorting by score puts every page above s^+ before every page below it, so the first k pages are the $\min(k, A)$ pages above s^+ followed, if $k > A$, by $k - A$ pages below it. □

Together, Propositions 1 and 2 say that no sampler trains against more possible false negatives than top-score, for every pool and every run; the comparison with ReSa2 needs no assumption about ReSa2.

Proposition 3 (The 7/75 share)**.** *Of all 7-page subsets of a set of 75 pages, the fraction that contains one fixed page is* $7/75$*; equivalently,* $75\binom{74}{6} = 7\binom{75}{7}$*. Hence* $P(\text{ReSa2 selects } i) = \frac{7}{75}\,P(i \text{ is among the 75 kept}$ *and equation 12 follows.*

Proof. A subset that contains the fixed page is the fixed page plus 6 of the other 74 pages, so there are $\binom{74}{6}$ of them out of $\binom{75}{7}$, and $\binom{74}{6}/\binom{75}{7} = 7/75$. The identity is checked by computation in Lean. □

The Lean statements are short enough to compare with the claims above:

```
theorem loss_le_sum_exp_neg_gap {n : ℕ} (sPos : ℝ)
    (s : Fin n → ℝ) :
    loss sPos s ≤ ∑ i, Real.exp (-(sPos - s i))

theorem any_sampler_le (sPos : Int)
    (pool sel : List Int) (h : sel.Sublist pool) :
    nAbove sPos sel
      ≤ min sel.length (nAbove sPos pool)

theorem topscore_is_max (sPos : Int) (k : Nat)
    (pool : List Int) :
    nAbove sPos (topscore sPos k pool)
      = min k (nAbove sPos pool)

theorem stage2_share :
    75 * choose 74 6 = 7 * choose 75 7
```

Table 1: Validation recall after 1,500 updates, mean ± standard deviation over three seeds. 16,692 mentions ranked against 5.87 million pages. Per-seed values are in Appendix A.

Sampler	Top-1	R@10	R@100
Released	42.6	77.7	93.1
In-batch	33.6 ± 1.9	69.6 ± 1.8	89.8 ± 1.3
Uniform	64.3 ± 0.2	92.2 ± 0.1	98.4 ± 0.1
Top-score	65.3 ± 1.2	87.2 ± 0.9	95.9 ± 0.6
ReSa2	**69.5** ± 0.3	**93.3** ± 0.1	**98.5** ± 0.1

What is not formalized: the derivative of equation 9 and the gradient of equation 10 are checked by hand above. The inclusion probability of stage 1 (successive sampling) is checked numerically: the exact enumeration for pools of up to 8 pages matches a simulation of the sampler to within 0.015. The training results of section 6 are empirical.

6 Experiments

6.1 Setup

I started each run from the released EVELINK model. I froze the page encoder and trained the mention encoder for 1,500 updates with a batch of 32 mentions (four micro-batches of 8), AdamW, a learning rate of 10^{-5} with 10% warm-up, gradient clipping at 1.0, and seven negatives per mention. Because the page encoder is frozen, the pool scores and similarities are fixed before training. I ran each of the four samplers with three seeds, twelve runs in total. Each run took about 56 minutes on one NVIDIA A10G.

Then I ranked all 5.87 million pages for each of the 16,692 validation mentions and computed Recall@K of equation 5 for $K = 1, 10, 100$. The metric I focus on is recall at ten: if the correct page appears anywhere in the first ten results, I count that mention as a hit.

Before the experiments I reproduced the released pipeline: the released retriever and reranker give 82.88% accuracy (equation 4) on the Wikipedia test set (paper: 82.87%) and 55.33% on the New York Times set (paper: 55.33%). The test set is untouched by everything below.

6.2 What changed in the rankings

Table 1 gives the results. The released retriever got 77.7 percent recall at ten. After fine-tuning, in-batch sampling got 69.6. Uniform sampling from the pool got 92.2. Taking the seven highest-scoring negatives got 87.2. ReSa2 got 93.3.

Most of the improvement came from drawing negatives from the retrieved pool. ReSa2 added 1.1 percentage points over uniform sampling at ten. The difference was larger at rank one. With uniform sampling, the correct page came first 64.3 percent of the time. With ReSa2, it came first 69.5 percent of the time. The spread between seeds is 0.1 to 0.3 points for both samplers, so the gap is about ten times the noise.

In-batch sampling finished below the released model. At this budget that is expected: seven random

Table 2: What each sampler selects: 500 validation mentions, 200 draws per mention, seven negatives per draw. The first column splits the seven negatives of one draw by the rank of their similarity to the correct page.

Sampler	Negatives by similarity rank top 25 / 25–75 / 75–300 / rest	Distinct pages (of 1,000)	Overlap of two draws	Score gap to correct page
Uniform	0.17 / 0.34 / 1.55 / 4.95	756	0.00	−13.3
Top-score	4.13 / 1.07 / 1.17 / 0.62	7	1.00	−1.4
ReSa2	2.15 / 3.98 / 0.87 / 0.00	84	0.05	−8.6

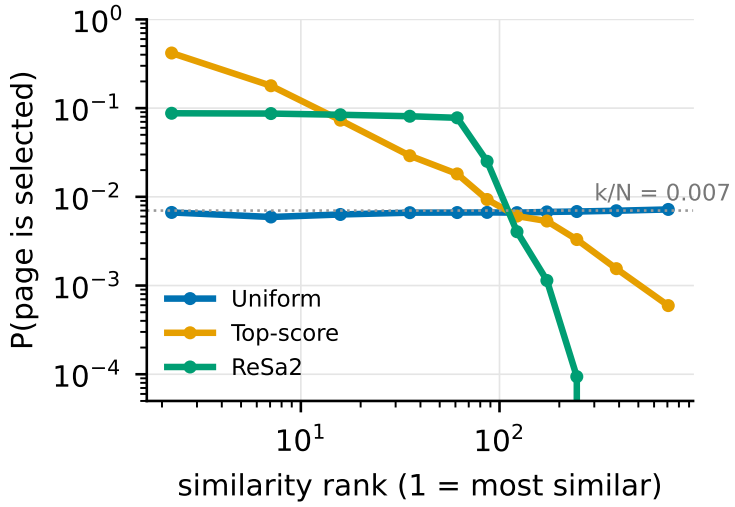


Figure 2: Inclusion probability of a pool page against its similarity rank to the correct page (300 validation mentions, 100 draws each, both axes logarithmic). Uniform is flat at k/N. Top-score is high for the most similar pages and falls slowly. ReSa2 is flat at about 0.09 over the 75 most similar pages and selects nothing beyond rank 300.

negatives per step give almost no gradient (equation 8), while the original training used many more negatives per step and ran for longer.

6.3 Which pages the samplers pick

I also wanted to see which pages the samplers actually picked. I drew negatives 200 times each for 500 validation mentions. Table 2 and Figure 2 give the result.

ReSa2 picked about six of its seven negatives from the 75 pages most similar to the correct page. For uniform sampling, the average was about half a page. Top-score gave me the same seven pages every time. Two ReSa2 draws shared very few pages (Jaccard overlap 0.05), even though both tended to draw from the same part of the pool. Over 200 draws uniform sampling touched 756 pool pages, ReSa2 84, top-score 7.

The score gap says how hard the negatives are. ReSa2's negatives score 8.6 points below the correct page on average, between uniform (−13.3, easy) and top-score (−1.4, hardest). Figure 2 shows the same thing as a profile: ReSa2's inclusion probability is flat over the 75 most similar pages and zero

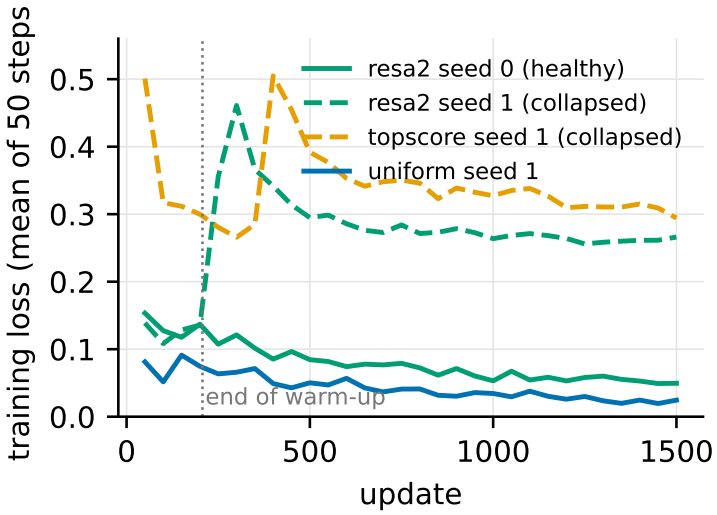


Figure 3: Training loss at learning rate 3×10^{-5} for a healthy run and the two collapsed runs. The collapsed runs jump once, near the end of the warm-up, and stay at about 0.3; the loss over eight pages looks modest while the ranking over millions of pages is noise.

beyond rank 300, while top-score concentrates on the first few pages, which are the near-duplicates.

6.4 What happened at the higher learning rate

At a learning rate of 3×10^{-5}, one ReSa2 run and one top-score run collapsed. Figure 3 shows the training loss: it jumped once, near the end of the warm-up, from about 0.13 to about 0.3, and stayed there. When I looked at the scores, I found that the mention encoder gave almost every page a score of about two. It gave roughly the same score to the correct page and to a random page; a healthy model gives about 550 to the correct page and 545 to a random one. The weights stayed finite. The loss over the eight training pages could still look modest while the ranking across millions of pages had fallen apart.

One possible explanation is in the gradient, equation 10. When a negative scores close to the correct page, it gets a substantial share of the training signal. If the two page vectors are also similar, the update depends on the small difference between them. My hypothesis is that repeated updates at the higher learning rate weakened the part of the mention vector that lined up with the page vectors. The scores are consistent with that explanation. Uniform and in-batch runs, whose negatives are far from the correct page, never jumped.

I tried training the collapsed model for another 1,500 updates. I tried ReSa2 negatives in one run and uniform negatives in another. Recall at ten stayed at zero in both. When I restarted from the released checkpoint and used a learning rate of 10^{-5}, all twelve runs trained without collapsing, and every sampler scored better than at the higher rate (Appendix A).

7 Conclusion

I started with the difference between the 1996 award and the 1998 award. The results make that example more specific for me. I want the retriever to see pages that are close enough to make it pay attention to the year. I also want it to see a different set of pages the next time it trains on that mention. In these runs, ReSa2 gave me that variety and put the correct page first more often than any of the other samplers I tested.

Limitations

All numbers are on the validation set. The test set is held back for the final version.

The page encoder is frozen, so the pool scores and similarities never change during training. A run that also trains the page encoder could behave differently, and its pools would have to be rebuilt as the encoder moves.

Each sampler ran at one budget (1,500 updates, 7 negatives) and two learning rates. The collapse appeared at one of them, and the gradient argument for it is a hypothesis, not a proof.

The pool is the released retriever's top 1,000 pages. A correct page outside the pool (2% of mentions) can never be sampled.

The Lean results cover the loss bound and the counting facts about possible false negatives. The derivative and the gradient are checked by hand.

How I used AI

The course asks for a description of how AI tools were used. I used two. Before the proposal, I used Codex to search for papers from researchers in the instructor's academic circle about linking documents, which led to Event Linking, and then for a more recent negative sampler, which led to ReSa2 and SimANS. It also wrote "learning versions" of the papers that explained them through the concepts of this course (conditional probability, indicator variables, expectation) with small examples; I read those first and the full papers second.

During the project I used Claude, through an agent that ran on my computer and on a cloud GPU. It set up the cloud account and the GPU machine. It changed the released EVELINK code so that the model fits a 24 GB GPU (a low-memory candidate store and mixed precision). From my descriptions it wrote the ReSa2 sampler and the hook that lets the training loop swap samplers, and it wrote the Lean proofs. It ran the 24 training runs and the recovery test, built the tables, and drew the figures. Along the way it fixed its own mistakes: an out-of-memory crash, a full disk, and a bug in the original validation code. The words are mine. I narrated each section, the agent turned the narration into plain technical English, and I corrected the result. The repository has an AI-usage log with one line per session, and I kept the prompts and outputs; I can produce the key ones on request.

References

[Yu et al.(2023)] Xiaodong Yu, Wenpeng Yin, Nitish Gupta, and Dan Roth. 2023. Event Linking: Grounding Event Mentions to Wikipedia. In *Proceedings of EACL*.

[Li et al.(2025)] Muyang Li, Zihan Wang, Sijia Chen, Yijun Chen, Jiayu Li, Yiming Qiao, Xinyi Li, and Bo Ji. 2025. ReSa2: A Two-Stage Retrieval-Sampling Algorithm for Negative Sampling in Dense Retrieval. In *Proceedings of ACML*, PMLR 304.

[Wu et al.(2020)] Ledell Wu, Fabio Petroni, Martin Josifoski, Sebastian Riedel, and Luke Zettlemoyer. 2020. Scalable Zero-shot Entity Linking with Dense Entity Retrieval. In *Proceedings of EMNLP*.

[Karpukhin et al.(2020)] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of EMNLP*.

[Xiong et al.(2021)] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2021. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In *Proceedings of ICLR*.

[Qu et al.(2021)] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2021. RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of NAACL*.

[Zhou et al.(2022)] Kun Zhou, Yeyun Gong, Xiao Liu, Wayne Xin Zhao, Yelong Shen, Anlei Dong, Jingwen Lu, Rangan Majumder, Ji-Rong Wen, and Nan Duan. 2022. SimANS: Simple Ambiguous Negatives Sampling for Dense Text Retrieval. In *Proceedings of EMNLP Industry Track*.

A Per-seed results

Top-1 / recall@10 / recall@100 on the validation set for every run. The learning rate 10^{-5} runs are the main results (Table 1); the 3×10^{-5} runs are the first attempt, in which two runs collapsed.

Table 3: Learning rate 10^{-5}.

Sampler	Seed 0	Seed 1	Seed 2
In-batch	36.0 / 71.7 / 91.2	31.4 / 67.4 / 88.1	33.4 / 69.7 / 90.1
Uniform	64.4 / 92.3 / 98.5	64.5 / 92.4 / 98.4	64.1 / 92.0 / 98.4
Top-score	64.4 / 86.5 / 95.6	64.6 / 86.7 / 95.4	67.0 / 88.5 / 96.7
ReSa2	70.0 / 93.2 / 98.4	69.4 / 93.3 / 98.5	69.3 / 93.4 / 98.6

The recovery test continued the collapsed top-score seed-1 model for 1,500 more updates: with ReSa2 negatives it scored 0.0 / 0.0 / 2.2, with uniform negatives 0.0 / 0.0 / 1.1.

Table 4: Learning rate 3×10^{-5} (first attempt).

Sampler	Seed 0	Seed 1	Seed 2
In-batch	24.4 / 57.6 / 82.9	22.7 / 56.4 / 83.1	25.0 / 60.7 / 85.0
Uniform	60.7 / 90.0 / 98.0	61.5 / 90.3 / 97.9	61.1 / 90.4 / 97.8
Top-score	55.2 / 78.3 / 90.1	0.0 / 0.1 / 0.9	57.8 / 80.6 / 92.3
ReSa2	68.3 / 92.0 / 98.0	0.0 / 0.1 / 1.2	67.5 / 92.1 / 98.1

B The Lean file

Two files in the project repository, Lean 4 v4.34.1, built with `lake build`. `lean/LossBound.lean` uses Mathlib (real numbers, exp, log); its proofs depend only on the axioms `propext`, `Classical.choice` and `Quot.sound`. `lean/FalseNegatives.lean` uses the core library only.

LossBound.lean

```
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib.Algebra.BigOperators.Fin

/-!
# Easy negatives add almost nothing to the loss

Uses Mathlib for the real numbers, ‘Real.exp‘, and ‘Real.log‘. Check with:
‘cd lean && lake build‘.

Setup. One training mention has a correct page with score ‘sPos‘ and ‘n‘
negatives with scores ‘s i‘. The retriever loss (softmax cross entropy) is

  L = -log (e^{sPos} / (e^{sPos} +  e^{s i})).

The gap of negative ‘i‘ is ‘g i = sPos - s i‘: how far it scores below the
correct page.

Results.
* ‘loss_eq_log_gaps‘: L = log (1 +  e^{-g i}). The loss depends only on
  the gaps.
* ‘loss_le_sum_exp_neg_gap‘: L ≤  e^{-g i}, because log (1 + x) ≤ x.
  So a negative with gap g adds at most e^{-g} to the bound.
* ‘loss_nonneg‘: L ≥ 0.
-/

open Real Finset

/-- The softmax cross-entropy loss of one mention: correct-page score ‘sPos‘,
negative scores ‘s 0, ..., s (n-1)‘. -/
noncomputable def loss {n : ℕ} (sPos : ℝ) (s : Fin n → ℝ) : ℝ :=
  -Real.log (Real.exp sPos / (Real.exp sPos + ∑ i, Real.exp (s i)))

/-- The loss written with the gaps ‘sPos - s i‘. -/
theorem loss_eq_log_gaps {n : ℕ} (sPos : ℝ) (s : Fin n → ℝ) :
    loss sPos s = Real.log (1 + ∑ i, Real.exp (-(sPos - s i))) := by
```

```
  have hpos : 0 < Real.exp sPos := Real.exp_pos sPos
  have hsum : 0 ≤ ∑ i, Real.exp (s i) :=
    Finset.sum_nonneg fun i _ => (Real.exp_pos (s i)).le
  -- e^{-(sPos - s i)} = e^{s i} / e^{sPos}
  have hgap : ∀ i, Real.exp (-(sPos - s i)) = Real.exp (s i) / Real.exp sPos := by
    intro i
    rw [neg_sub, Real.exp_sub]
  -- 1 +  e^{-g i} = (e^{sPos} +  e^{s i}) / e^{sPos}
  have hrw : 1 + ∑ i, Real.exp (-(sPos - s i))
      = (Real.exp sPos + ∑ i, Real.exp (s i)) / Real.exp sPos := by
    rw [eq_div_iff hpos.ne', add_mul, one_mul, Finset.sum_mul]
    congr 1
    refine Finset.sum_congr rfl fun i _ => ?_
    rw [hgap i, div_mul_cancel₀ _ hpos.ne']
  rw [hrw, loss,  Real.log_inv, inv_div]

/-- The loss is never negative. -/
theorem loss_nonneg {n : ℕ} (sPos : ℝ) (s : Fin n → ℝ) : 0 ≤ loss sPos s := by
  rw [loss_eq_log_gaps]
  apply Real.log_nonneg
  have : 0 ≤ ∑ i, Real.exp (-(sPos - s i)) :=
    Finset.sum_nonneg fun i _ => (Real.exp_pos _).le
  linarith

/-- Main bound: L ≤  e^{-g i}. Each negative adds at most e^{-g} to it. -/
theorem loss_le_sum_exp_neg_gap {n : ℕ} (sPos : ℝ) (s : Fin n → ℝ) :
    loss sPos s ≤ ∑ i, Real.exp (-(sPos - s i)) := by
  rw [loss_eq_log_gaps]
  set x := ∑ i, Real.exp (-(sPos - s i)) with hx
  have hx0 : 0 ≤ x := Finset.sum_nonneg fun i _ => (Real.exp_pos _).le
  -- log y ≤ y - 1 for y > 0, with y = 1 + x
  have := Real.log_le_sub_one_of_pos (by linarith : (0 : ℝ) < 1 + x)
  linarith

/-- Corollary: if every negative scores at least ‘g‘ below the correct page,
then L ≤ n · e^{-g}. -/
theorem loss_le_of_gap_ge {n : ℕ} (sPos g : ℝ) (s : Fin n → ℝ)
    (hg : ∀ i, g ≤ sPos - s i) :
    loss sPos s ≤ n * Real.exp (-g) := by
  calc loss sPos s ≤ ∑ i, Real.exp (-(sPos - s i)) := loss_le_sum_exp_neg_gap sPos s
    _ ≤ ∑ _i : Fin n, Real.exp (-g) :=
        Finset.sum_le_sum fun i _ => Real.exp_le_exp.mpr (by linarith [hg i])
    _ = n * Real.exp (-g) := by simp
```

FalseNegatives.lean

```
/-!
# Small Lean model of the false-negative claim in the proposal

Core Lean 4 only (no Mathlib). Check with:  cd lean && lake build

Setup. A page is its score (an integer, to stay simple). The correct page has
score ‘sPos‘. A "possible false negative" is a page that scores above ‘sPos‘.
```

```
A sampler returns a list of pages taken from the pool.

Main result (‘topscore_is_max‘): the top-score rule selects ‘min k A‘ possible
false negatives, where ‘A‘ is how many the pool has, and NO sampler of ‘k‘
pages can select more (‘any_sampler_le‘). So "ReSa2 selects fewer of them than
top-score, in expectation" is true for every sampler, every pool, and every
run. It needs no condition on ReSa2.
-/

/-- ‘s‘ scores above the correct page. -/
def above (sPos s : Int) : Bool := decide (sPos < s)

/-- Number of possible false negatives in a list of pages. -/
def nAbove (sPos : Int) (l : List Int) : Nat := l.countP (above sPos)

/-! ## 1. An upper bound for every sampler -/

/-- Any selection taken from the pool (each page used at most once) has at most
‘min (its size) A‘ possible false negatives. (Write the selected pages in
pool order; then "each page used at most once" means ‘sel‘ is a sublist.) -/
theorem any_sampler_le (sPos : Int) (pool sel : List Int) (h : sel.Sublist pool) :
    nAbove sPos sel ≤ min sel.length (nAbove sPos pool) :=
  Nat.le_min.mpr ⟨List.countP_le_length, h.countP_le⟩

/-! ## 2. The top-score rule reaches the bound

Sorting by score puts every page above ‘sPos‘ before every page below it.
So, for counting, top-score is: pages above first, then the others, take ‘k‘. -/

def topscore (sPos : Int) (k : Nat) (pool : List Int) : List Int :=
  (pool.filter (above sPos) ++ pool.filter (fun s => !above sPos s)).take k

/-- Taking ‘k‘ from (all-good list ++ no-good list) gives ‘min k (good length)‘ goods. -/
theorem countP_take_append (p : Int → Bool) (a b : List Int)
    (ha : ∀ x ∈ a, p x = true) (hb : ∀ x ∈ b, p x = false) :
    ∀ k, ((a ++ b).take k).countP p = min k a.length := by
  induction a with
  | nil =>
    intro k
    simp only [List.nil_append, List.length_nil, Nat.min_zero]
    exact List.countP_eq_zero.mpr fun x hx => by
      simp [hb x (List.mem_of_mem_take hx)]
  | cons x xs ih =>
    intro k
    cases k with
    | zero => simp
    | succ k =>
      have hx : p x = true := ha x (by simp)
      have ih' := ih (fun y hy => ha y (by simp [hy])) k
      simp only [List.cons_append, List.take_succ_cons, List.countP_cons, hx,
        ite_true, List.length_cons, ih']
      omega

theorem topscore_is_max (sPos : Int) (k : Nat) (pool : List Int) :
```

```
    nAbove sPos (topscore sPos k pool) = min k (nAbove sPos pool) := by
  unfold nAbove topscore
  rw [countP_take_append (above sPos) _ _ (fun x hx => (List.mem_filter.mp hx).2)
      (fun x hx => by simpa using (List.mem_filter.mp hx).2)]
  rw [List.countP_eq_length_filter]

/-! ## 3. Stage 2 of ReSa2 picks 7 of the top 75 uniformly

Counting fact: of all 7-page subsets of 75 pages, the share that contains one
fixed page is 7/75. So P(page selected) = (7/75) · P(page is in the top 75),
which is at most (7/75) · P(page is kept by stage 1). -/

/-- Binomial coefficient. Step i: C(n, i+1) = C(n, i) · (n − i) / (i + 1),
    and each division is exact. -/
def choose (n k : Nat) : Nat :=
  (List.range k).foldl (fun c i => c * (n - i) / (i + 1)) 1

/-- Subsets that contain the fixed page: choose the other 6 from 74.
    `75 · C(74,6) = 7 · C(75,7)`, that is, the share is 7/75. -/
theorem stage2_share : 75 * choose 74 6 = 7 * choose 75 7 := by decide

/-! ## Example with numbers -/

-- Pool: correct page at 0; three pages above it.
#eval nAbove 0 [5, -3, 1, -8, 2, -1]                 -- 3
#eval topscore 0 2 [5, -3, 1, -8, 2, -1]             -- [5, 1]
#eval nAbove 0 (topscore 0 7 [5, -3, 1, -8, 2, -1])  -- 3 = min 7 3
```